

Unity Graphics Programming - Volume 4

Contents

Preface - Unity Graphics Programming Volume 4	3
About This Book	3
Source Code Repository	3
Difficulty Levels	3
About IndieVisualLab	3
Recommended Execution Environment	3
Feedback and Requests	3
Key Takeaways	4
Chapter 1: GPU-Based Space Colonization Algorithm	4
Introduction	4
Algorithm Overview	4
Implementation	5
Rendering	9
Application: Skinned Animation Integration	10
Key Parameters	11
Key Takeaways	12
References	12
Chapter 2: Limit Sets of Kleinian Groups	12
Introduction	12
Circle Inversion	12
Mobius Transformations	13
Schottky Groups and Kleinian Groups	14
Limit Sets	14
Jos Leys' Algorithm	14
Implementation	15
Adding Circle Inversion for Animation	17
Key Parameters	18
Key Takeaways	18
References	18
Chapter 3: GPU-Based Cloth Simulation	18
Introduction	18
Algorithm Overview	19
Implementation	20
Parameters Guide	25
Ping-Pong Buffering	25
Key Takeaways	26
Future Directions	26
References	26

Chapter 4: StarGlow	26
Introduction	26
Overview	26
STEP 1: Generate Brightness Image	27
STEP 2: Apply Directional Blur	27
STEP 2.5: Composite Multiple Directions	29
STEP 3: Final Composite	30
STEP 4: Resource Cleanup	31
Parameters Summary	31
Performance Considerations	32
Variations and Extensions	32
Key Takeaways	32
References	32
Chapter 5: Triangulation by Ear Clipping	32
Introduction	32
Simple Polygon Triangulation	33
The Ear Clipping Algorithm	33
Triangulation Process	33
Polygons with Holes	34
Nested Polygon Hierarchies	34
Implementation	35
Geometry Utilities Summary	38
Key Takeaways	38
Applications	38
Performance Note	39
References	39
Chapter 6: Tessellation & Displacement	39
Introduction	39
What is Tessellation?	39
Tessellation Pipeline Stages	39
Surface Shader Implementation	40
Vertex/Fragment Shader Implementation	42
Application Example: Fluid Displacement	45
Performance Considerations	45
Key Takeaways	45
Best Practices	45
References	46
Chapter 7: Poisson Disk Sampling	46
Introduction	46
What is Poisson Disk Sampling?	46
Fast Poisson Disk Sampling Algorithm	46
Algorithm Steps	47
Algorithm Summary	50
Complexity Analysis	50
Visualization	50
Parameter Guidelines	51
Limitations	51
Key Takeaways	51
Applications in Graphics	51
References	51

Preface - Unity Graphics Programming Volume 4

Series: Unity Graphics Programming **Volume:** 4 **Publisher:** IndieVisualLab

About This Book

This book is the fourth volume of the “Unity Graphics Programming” series, which explains graphics programming techniques using Unity.

In this series, we cover a variety of topics selected based on the authors’ interests, explaining both introductory content for beginners along with applications, as well as tips for intermediate and advanced readers.

Source Code Repository

The source code explained in each chapter is available on the GitHub repository: - **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4>

You can follow along with this book while running the code locally.

Difficulty Levels

The difficulty varies by chapter. Depending on your knowledge level, some content may feel too easy or too difficult. We recommend reading articles on topics that interest you according to your own knowledge level.

For those who regularly work with graphics programming professionally, we hope this book helps expand your repertoire of effects. For students interested in visual coding who may have experience with Processing or openFrameworks but still find 3D CG intimidating, we hope this serves as an introduction to Unity and helps you discover the expressive power of 3D CG and how to get started with development.

About IndieVisualLab

IndieVisualLab is a circle founded by colleagues (and former colleagues) from the same company. At work, we use Unity to create content programming for exhibition works in the category commonly known as media art. This represents a different flavor of Unity usage compared to game development.

Throughout this book, you may find scattered knowledge that is useful when utilizing Unity in exhibition works.

Recommended Execution Environment

Some content explained in this book uses Compute Shaders and Geometry Shaders. We recommend an execution environment that supports DirectX 11, though some chapters complete their content entirely with CPU-side programming (C#).

Due to differences in environments, the behavior of the published sample code may not work correctly. Please report issues to the GitHub repository or adjust the code as needed.

Feedback and Requests

If you have feedback, concerns, or other requests regarding this book (such as “I’d like to read an explanation about X”), please let us know via the Web form or email (lab.indievisual@gmail.com).

Figure 1: Skinned Animation Scene

Key Takeaways

- **Volume 4** continues the series tradition of practical graphics programming techniques
- **Source code** is available for hands-on learning
- **Mixed difficulty levels** accommodate both beginners and advanced readers
- **Media art focus** provides unique perspective on Unity usage beyond games
- **DirectX 11 recommended** for full shader support, though some chapters are CPU-only

Chapter 1: GPU-Based Space Colonization Algorithm

Author: Nakamura **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** SpaceColonization

Introduction

This chapter introduces the Space Colonization Algorithm - a technique for generating branching structures that follow point clouds - along with its GPU implementation and an application example combining it with skinned animation.

What is the Space Colonization Algorithm?

The Space Colonization Algorithm was developed by Adam et al. as a tree modeling technique. It generates branching structures from a given point cloud with these key characteristics:

- **Balanced distribution** - Branches spread out naturally without clustering too densely
- **Shape control** - Branch placement is determined by initial point cloud positioning, making shapes controllable
- **Simple parameters** - Branch density can be controlled with simple parameters

Algorithm Overview

The algorithm consists of six main steps:

1. **Setup** - Initialization
2. **Search** - Find influencing Attractions
3. **Attract** - Pull branches toward Attractions
4. **Connect** - Generate new Nodes and connect to existing Nodes
5. **Remove** - Delete Attractions within kill distance
6. **Grow** - Node growth

Step 1: Setup - Initialization

During initialization, prepare a point cloud as **Attractions** (seed points for branches). Place one or more **Nodes** (branch points) within this Attraction field. These initially placed Nodes become the starting points for branches.

In the diagrams: - Circular points represent Attractions - Square points represent Nodes

Step 2: Search - Finding Nearest Nodes

For each Attraction, search for the nearest Node within the **influence distance**.

Step 3: Attract - Determining Branch Direction

For each Node with Attractions within influence range: 1. Calculate the direction to extend the branch 2. Find the point at **growth distance** along that direction 3. This point becomes a **Candidate** for a new Node

Step 4: Connect - Extending Branches

Generate new Nodes at Candidate positions and connect them to original Nodes with Edges, extending the branch structure.

Step 5: Remove - Cleaning Up Attractions

Delete Attractions within the **kill distance** from any Node.

Step 6: Grow - Node Growth

Grow the Nodes and return to Step 2.

Implementation

Resource Structures

The Space Colonization Algorithm requires four element types that can increase/decrease:

Attraction (Seed Points)

```
public struct Attraction {  
    public Vector3 position; // Position  
    public int nearest;      // Nearest Node index  
    public uint found;       // Whether a nearby Node was found  
    public uint active;      // Whether this Attraction is active (1=active, 0=deleted)  
}
```

The active flag allows soft deletion of Attractions without actually removing them from the buffer.

Node (Branch Points)

```
public struct Node {  
    public Vector3 position; // Position  
    public float t;          // Growth rate (0.0 ~ 1.0)  
    public float offset;     // Distance from Root (Node depth)  
    public float mass;       // Mass  
    public int from;         // Index of parent Node  
    public uint active;      // Whether this Node is active  
}
```

Nodes use two buffers: 1. Buffer for actual Node data 2. Object pool buffer managing indices of inactive Nodes (Append/ConsumeStructuredBuffer)

Candidate (New Node Candidates)

```
public struct Candidate {  
    public Vector3 position; // Position  
    public int node;        // Index of source Node  
}
```

Edge (Connections Between Nodes)

```
public struct Edge {  
    public int a, b; // Indices of the two connected Nodes  
}
```

ComputeShader Implementation

Setup Kernel The Setup kernel initializes the object pool by filling it with indices and marking all Nodes as inactive:

```
void Setup (uint3 id : SV_DispatchThreadID)  
{  
    uint idx = id.x;  
    uint count, stride;  
    _Nodes.GetDimensions(count, stride);  
    if (idx >= count)  
        return;  
  
    _NodesPoolAppend.Append(idx);  
  
    Node n = _Nodes[idx];  
    n.active = false;  
    _Nodes[idx] = n;  
}
```

Seed Kernel The Seed kernel creates initial Nodes at specified positions:

```
void Seed (uint3 id : SV_DispatchThreadID)  
{  
    uint idx = id.x;  
    uint count, stride;  
    _Seeds.GetDimensions(count, stride);  
    if (idx >= count)  
        return;  
  
    Node n;  
    uint i = CreateNode(n); // Get index from object pool  
  
    n.position = _Seeds[idx];  
    n.t = 1;  
    n.offset = 0;  
    n.from = -1;  
    n.mass = lerp(_MassMin, _MassMax, nrand(id.xy));  
    _Nodes[i] = n;  
}
```

Search Kernel The Search kernel finds the nearest Node within influence distance for each Attraction:

```
void Search (uint3 id : SV_DispatchThreadID)
{
    uint idx = id.x;
    // ... bounds checking ...

    Attraction attr = _Attractions[idx];
    attr.found = false;

    if (attr.active)
    {
        float min_dist = _InfluenceDistance;
        uint nearest = -1;

        // Loop through all Nodes
        for (uint i = 0; i < count; i++)
        {
            Node n = _Nodes[i];
            if (n.active)
            {
                float3 dir = attr.position - n.position;
                float d = length(dir);
                if (d < min_dist)
                {
                    min_dist = d;
                    nearest = i;
                    attr.found = true;
                    attr.nearest = nearest;
                }
            }
        }
        _Attractions[idx] = attr;
    }
}
```

Attract Kernel The Attract kernel calculates candidate positions for new Nodes:

```
void Attract (uint3 id : SV_DispatchThreadID)
{
    // ... setup ...

    Node n = _Nodes[idx];

    if (n.active && n.t >= 1.0)
    {
        float3 dir = (0.0).xxx;
        uint counter = 0;

        // Accumulate direction vectors to all influencing Attractions
        for (uint i = 0; i < count; i++)
        {
            Attraction attr = _Attractions[i];
            if (attr.active && attr.found && attr.nearest == idx)
            {
```

```

        float3 dir2 = (attr.position - n.position);
        dir += normalize(dir2);
        counter++;
    }
}

if (counter > 0)
{
    Candidate c;
    dir = dir / counter; // Average direction
    c.position = n.position + (dir * _GrowthDistance);
    c.node = idx;
    _CandidatesAppend.Append(c);
}
}
}

```

Connect Kernel The Connect kernel creates new Nodes from Candidates and connects them with Edges:

```

void Connect (uint3 id : SV_DispatchThreadID)
{
    uint idx = id.x;
    if (idx >= _ConnectCount)
        return;

    Candidate c = _CandidatesConsume.Consume();

    Node n1 = _Nodes[c.node];
    Node n2;

    uint idx2 = CreateNode(n2);
    n2.position = c.position;
    n2.offset = n1.offset + 1.0; // Depth = parent + 1
    n2.from = c.node;
    n2.mass = lerp(_MassMin, _MassMax, nrand(float2(c.node, idx2)));

    _Nodes[c.node] = n1;
    _Nodes[idx2] = n2;

    CreateEdge(c.node, idx2);
}

```

Remove Kernel The Remove kernel deactivates Attractions within kill distance of any Node:

```

void Remove(uint3 id : SV_DispatchThreadID)
{
    // ... setup ...

    Attraction attr = _Attractions[idx];
    if (!attr.active)
        return;

    for (uint i = 0; i < count; i++)
    {

```



```

    Node n = _Nodes[i];
    if (n.active)
    {
        float d = distance(attr.position, n.position);
        if (d < _KillDistance)
        {
            attr.active = false;
            _Attractions[idx] = attr;
            return;
        }
    }
}
}

```

Grow Kernel The Grow kernel increases the growth rate of each Node:

```

void Grow (uint3 id : SV_DispatchThreadID)
{
    // ... setup ...

    Node n = _Nodes[idx];
    if (n.active)
    {
        // Mass parameter randomizes growth speed per Node
        n.t = saturate(n.t + _DT * n.mass);
        _Nodes[idx] = n;
    }
}

```

Rendering

Line Topology Rendering

The simplest approach uses Line Mesh topology with GPU instancing:

```

protected Mesh BuildSegment()
{
    var mesh = new Mesh();
    mesh.vertices = new Vector3[2] { Vector3.zero, Vector3.up };
    mesh.uv = new Vector2[2] { new Vector2(0f, 0f), new Vector2(0f, 1f) };
    mesh.SetIndices(new int[2] { 0, 1 }, MeshTopology.Lines, 0);
    return mesh;
}

```

The vertex shader controls Edge length based on Node growth rate:

```

v2f vert(appdata IN, uint iid : SV_InstanceID)
{
    v2f OUT;

    Edge e = _Edges[iid];
    Node a = _Nodes[e.a];
    Node b = _Nodes[e.b];
}

```

```

float3 ap = a.position;
float3 bp = b.position;
float3 dir = bp - ap;

// Scale edge length by Node b's growth rate
bp = ap + normalize(dir) * length(dir) * b.t;

// Vertex ID 0 = node a, 1 = node b
float3 position = lerp(ap, bp, IN.vid);

OUT.position = UnityObjectToClipPos(float4(position, 1));
OUT.alpha = (a.active && b.active) && (iid < _EdgesCount);

return OUT;
}

```

Geometry Shader Rendering

For thicker lines, a Geometry Shader can convert Line segments to capsule shapes:

```

[maxvertexcount(64)]
void geom(line v2g IN[2], inout TriangleStream<g2f> OUT) {
    // Calculate tangent, normal, binormal
    float3 t = normalize(p1.position - p0.position);
    float3 n = normalize(p0.viewDir);
    float3 bn = cross(t, n);
    n = cross(t, bn);

    // Build capsule side
    for (uint i = 0; i < cols; i++) {
        float r = (i * cols_inv) * UNITY_TWO_PI;
        float s, c;
        sincos(r, s, c);
        float3 normal = normalize(n * c + bn * s);
        // ... emit vertices ...
    }

    // Build capsule end caps (hemispheres)
    // ... hemisphere construction ...
}

```

Application: Skinned Animation Integration

This advanced application combines Space Colonization with skinned mesh animation, creating branches that follow animated model shapes.

Overview

1. Prepare an animated model
2. Generate a point cloud filling the model's volume (Attractions)
3. Add Bone information to Attractions and Nodes
4. Apply Bone transformations (skinning) to Node positions

Modified Structures

```
public struct SkinnedAttraction {
    public Vector3 position;
    public int bone;           // Bone index
    public int nearest;
    public uint found;
    public uint active;
}

public struct SkinnedNode {
    public Vector3 position;
    public Vector3 animated;  // Position after skinning
    public int index0;        // Bone index
    public float t;
    public float offset;
    public float mass;
    public int from;
    public uint active;
}
```

Volume Sampling

The VolumeSampler package generates point clouds filling mesh volumes using: 1. GPU-based voxelization (from Volume 2's "Real-Time GPU-Based Voxelizer") 2. Poisson Disk Sampling to create evenly distributed points

Skinning Implementation

Each Node receives bone information from its nearest Attraction. The Animate kernel applies bone transformations:

```
void Animate (uint3 id : SV_DispatchThreadID)
{
    // ... setup ...

    SkinnedNode node = _Nodes[idx];
    if (node.active)
    {
        // Apply skinning transformation
        float4x4 bind = _BindPoses[node.index0];
        float4x4 m = _BoneMatrices[node.index0];
        node.animated = mul(mul(m, bind), float4(node.position, 1)).xyz;
        _Nodes[idx] = node;
    }
}
```

The rendering shader then uses the animated position instead of the original position.

Key Parameters

The algorithm is controlled by three main parameters:

Parameter	Description	Effect
Influence Distance	Range within which Attractions affect Nodes	Larger = more widely spread branches
Growth Distance	Length each branch segment grows	Larger = longer segments, sparser structure
Kill Distance	Range within which Attractions are removed	Larger = fewer, thicker branches

Advanced Variations

- **Localized parameters** - Vary parameters spatially for different densities in different areas
- **Time-varying parameters** - Animate parameters for evolving structures
- **Dynamic Attractions** - Add Attractions during execution for more complex patterns

Key Takeaways

1. **Space Colonization** generates organic branching structures from point clouds
2. **GPU implementation** enables real-time generation using Append/Consume buffers
3. **Object pooling** on GPU manages dynamic Node/Edge creation
4. **Three key parameters** (influence, growth, kill distance) control branch density
5. **Skinned animation integration** allows branches to follow animated meshes
6. **Geometry Shader rendering** creates volumetric branch visualization

References

- Modeling Trees with a Space Colonization Algorithm - <http://algorithmicbotany.org/papers/colonization.egwnp2007.la>
- Algorithmic Design with Houdini - <https://vimeo.com/305061631#t=1500s>

Chapter 2: Limit Sets of Kleinian Groups

Author: Fuqunaga **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** KleinianGroup

Introduction

This chapter introduces how to draw the limit sets of Kleinian groups using shaders and animate the resulting fractal figures. While fractal animations typically involve zooming in and out to observe self-similar patterns, this technique also produces characteristic movements where straight lines and circles smoothly transition into each other.

Circle Inversion

Let's start with an introduction to geometric inversion. You're likely familiar with reflection across a line (line symmetry) and reflection through a point (point symmetry). There also exists **circle inversion** - an operation that exchanges the interior and exterior of a circle across the 2D plane.

Mathematical Definition

For a circle with center O and radius r, inversion maps a point P to a point P' such that:

$$|OP| \cdot |OP'| = r^2$$

where OP and OP' point in the same direction.

Near the circle's circumference, the inside and outside exchange like a distorted line symmetry. Infinitely far points exchange with the circle's center.

Inverting Lines

When inverting a line outside the circle: - Points near the circle map to just inside the circumference - Points far from the circle map closer to the center - The result appears as a small circle inside the original circle

If we consider a straight line as a circle with infinite radius, then circle inversion exchanges circles inside and outside the boundary circle.

Formula for Unit Circle Inversion

On the complex plane, inversion through the unit circle is expressed as:

$$z \rightarrow \frac{1}{\bar{z}}$$

where z is a complex number and z-bar is its complex conjugate.

Expanding this:

$$z \rightarrow \frac{1}{\bar{z}} = \frac{z}{|z|^2}$$

This shows that inversion scales z by the inverse of its squared magnitude.

Complex Plane Operations

On the complex plane: - **Addition** = Translation - **Multiplication** = Rotation (and scaling) - **Division** (including inversion) = A new type of transformation

Mobius Transformations

Generalizing complex plane transformations to include division gives us the **Mobius transformation**:

$$z \rightarrow \frac{az + b}{cz + d}$$

where a, b, c, d are all complex numbers.

Schottky Groups and Kleinian Groups

We can create fractal figures by repeatedly applying Mobius transformations.

Construction

1. Prepare four non-intersecting circles: D_A, D_a, D_B, D_b
2. Create Mobius transformation **a** that maps the exterior of D_A to the interior of D_a (and vice versa)
3. Create Mobius transformation **b** that maps D_B exterior to D_b interior
4. Also define inverse transformations **A** and **B**

The set of all possible compositions of these four transformations (**a**, **A**, **b**, **B**) in any order (e.g., **aaBAbbaB**) is called a **Schottky group** generated by **a** and **b**.

Kleinian groups are the more general term for discrete groups of Mobius transformations.

Limit Sets

Repeatedly applying Schottky group transformations produces circles within circles within circles, continuing infinitely. This infinite collection of nested circles is called the **limit set** of the Schottky group.

Drawing this limit set is the goal of this chapter.

Jos Leys' Algorithm

Overview

Naively implementing limit set rendering would be computationally infeasible due to infinite transformation combinations. Fortunately, Jos Leys has published an efficient algorithm for this purpose.

The Transformations

We use two Mobius transformations:

$$a : z \rightarrow \frac{tz - i}{-iz}$$

$$b : z \rightarrow z + 2$$

where t is a complex number $u + iv$. Varying t changes the figure's shape.

Analyzing Transformation **a**

Expanding transformation **a**:

$$a : z \rightarrow \frac{tz - i}{-iz} = \frac{1}{z} + (-v + iu)$$

Further:

$$\frac{1}{z} = \frac{x - iy}{|z|^2}$$

Therefore transformation a consists of: 1. Unit circle inversion 2. Negate the y-component (reflection) 3. Translate by (-v, u)

The Limit Set Structure

The limit set forms a band-shaped figure of connected large and small circles with these properties:

- Bounded by $0 \leq y \leq u$
- Horizontally repeating parallelograms between Line1 and Line2
- Line1 passes through (1, 0) and (1-v, u)
- Line2 passes through (-1, 0) and (-1-v, u)
- Line3 divides the figure, with upper and lower halves being point-symmetric about $(-v/2, u/2)$

The Algorithm

To determine if a point belongs to the limit set:

1. If $y < 0$ or $y > u$, the point is **outside** the band
2. If right of Line1 or left of Line2, translate to center parallelogram
3. If above Line3, reflect through center point to below Line3
4. Apply transformation a
5. If $y < 0$ after transformation, the point is **in the limit set**
6. If the point equals the position from 2 iterations ago, it's **not in the limit set**
7. Otherwise, return to step 2

The key insight: applying transformation a to a point inside the largest circle (which touches $y=0$) results in $y < 0$. Therefore:

If applying transformation a yields $y < 0$, the point is inside the largest circle, thus inside the limit set.

Implementation

C# Script

```
private void OnRenderImage(RenderTexture source, RenderTexture destination)
{
    material.SetColor("_HitColor", hitColor);
    material.SetColor("_BackColor", backColor);
    material.SetInt("_Iteration", iteration);
    material.SetFloat("_Scale", scale);
    material.SetVector("_Offset", offset);
    material.SetVector("_Circle", circle);

    Vector2 uv = kleinUV;
    if (useUVAnimation)
    {
        uv = new Vector2(
            animKleinU.Evaluate(time),
            animKleinV.Evaluate(time)
        );
    }
}
```

```

    }
    material.SetVector("_KleinUV", uv);
    Graphics.Blit(source, destination, material, pass);
}

```

Fragment Shader

```

fixed4 frag (v2f_img i) : SV_Target
{
    float2 pos = i.uv;
    float aspect = _ScreenParams.x / _ScreenParams.y;
    pos.x *= aspect;
    pos += _Offset;
    pos *= _Scale;

    bool hit = josKleinian(pos, _KleinUV);
    return hit ? _HitColor : _BackColor;
}

```

The josKleinian Function

```

bool josKleinian(float2 z, float2 t)
{
    float u = t.x;
    float v = t.y;

    float2 lz = z + (1).xx;    // Two iterations ago
    float2 llz = z + (-1).xx;  // One iteration ago

    for (uint i = 0; i < _Iteration; i++)
    {
        // Wrap to center parallelogram
        float offset_x = abs(v) / u * z.y;
        z.x += offset_x;
        z.x = wrap(z.x, 2, -1);
        z.x -= offset_x;

        // If above Line3, reflect through center point
        float separate_line = u * 0.5
            + sign(v) * (2 * u - 1.95) / 4 * sign(z.x + v * 0.5)
            * (1 - exp(-(7.2 - (1.95 - u) * 15) * abs(z.x + v * 0.5)));

        if (z.y >= separate_line)
        {
            z = float2(-v, u) - z;
        }

        // Apply transformation a
        z = TransA(z, t);

        // Hit detection
        if (z.y < 0) { return true; }

        // Two-cycle detection (not in limit set)
    }
}

```



```

        if (length(z - llz) < 1e-6) { break; }

        llz = lz;
        lz = z;
    }

    return false;
}

```

Transformation a Implementation

```

float2 TransA(float2 z, float2 t)
{
    // z -> (x - iy) / |z|^2 + (-v + iu)
    return float2(z.x, -z.y) / dot(z, z) + float2(-t.y, t.x);
}

```

Wrap Function

```

float wrap(float x, float width, float left_side)
{
    x -= left_side;
    return (x - width * floor(x / width)) + left_side;
}

```

Adding Circle Inversion for Animation

To create more interesting animations, we can apply an additional circle inversion before the limit set calculation:

```

float2 circleInverse(float2 pos, float2 center, float radius)
{
    float2 p = pos - center;
    p = (p * radius) / dot(p, p);
    p += center;
    return p;
}

fixed4 frag_circle_inverse(v2f_img i) : SV_Target
{
    float2 pos = i.uv;
    float aspect = _ScreenParams.x / _ScreenParams.y;
    pos.x *= aspect;
    pos *= _Scale;
    pos += _Offset;

    // Anti-aliasing through multi-sampling
    int sample_num = 10;
    float4 sum;
    for (int i = 0; i < sample_num; ++i)
    {
        float2 offset = rand2n(pos, i) * (1 / _ScreenParams.y) * 3;
        float2 p = circleInverse(pos + offset, _Circle.xy, _Circle.w);
    }
}

```

```

        sum += calc_color(p);
    }

    return sum / sample_num;
}

```

Circle inversion: - Exchanges the infinite exterior with the circle's interior - Preserves circles as circles - Creates unexpected, fascinating patterns when applied to the limit set

Moving the inversion circle or changing its radius produces mesmerizing fractal animations.

Key Parameters

Parameter	Recommended Value	Description
t (u, v)	(2, 0) standard (~1.94, ~0.02)	Controls figure shape Produces interesting complex shapes
_Iteration	~20-50	Maximum algorithm iterations
_Scale	Variable	Zoom level
_Offset	Variable	Pan position

Key Takeaways

1. **Circle inversion** exchanges interior/exterior regions, preserving circular shapes
2. **Mobius transformations** generalize complex plane operations including inversion
3. **Kleinian groups** generate fractal limit sets through repeated Mobius transformations
4. **Jos Leys' algorithm** efficiently determines limit set membership
5. **Two-cycle detection** identifies points outside the limit set
6. **Additional circle inversion** creates dramatic animation effects

References

- **Indra's Pearls** - The definitive resource on this topic (advanced mathematical content)
- **Mathe Vital** - Accessible introduction extracting key concepts from Indra's Pearls
- **Jos Leys' algorithm paper** - http://www.josleys.com/articles/Kleinian%20escape-time_3.pdf
- **Jos Leys' Shadertoy** - <https://www.shadertoy.com/user/JosLeys>
- **Morph by soma_arc** - <https://www.shadertoy.com/view/MIGfDG> (Tokyo Demo Fest 2018)

Chapter 3: GPU-Based Cloth Simulation

Author: Oishi **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** GPUClothSimulation

Introduction

Simulating deformable planar objects like flags and clothing under external forces is called **Cloth Simulation**. This is essential for animation in CG, and extensive research exists in this field.

While Unity already has built-in cloth simulation, this chapter introduces simple cloth simulation theory and GPU implementation for the purposes of learning parallel computation and understanding simulation properties and parameters.

Algorithm Overview

Mass-Spring System

Objects like springs, rubber, and cushions that deform under force and return to original shape are called **elastic bodies**. Since elastic bodies cannot be represented by a single position or orientation, we represent them as points connected by links:

- **Mass points** - Dimensionless points with mass
- **Springs** - Connections between mass points with elastic properties

Simulating elastic bodies by calculating spring stretching/compression is called a **Mass-Spring System**. When applied to a 2D array of mass points for simulating flags and clothing, it's called **Cloth Simulation**.

Spring Force

Each spring applies force to connected mass points according to:

$$F_{spring} = -k(l - l_0) - bv$$

Where: - **l** = Current spring length (distance between connected mass points) - **l₀** = Natural length (rest length with no load) - **k** = Spring stiffness constant - **v** = Mass point velocity - **b** = Damping constant

This equation means springs always try to return to their natural length. Greater deviation from natural length produces greater force, with velocity-proportional damping.

Spring Structure

The simulation uses two types of springs:

1. **Structure Springs** - Connect horizontally and vertically adjacent points
2. **Shear Springs** - Connect diagonally adjacent points (prevent extreme diagonal deformation)

Each mass point connects to up to 12 neighboring points: - 8 immediate neighbors (structure + shear) - 4 second-ring neighbors for additional stability

Verlet Integration

This simulation uses **Verlet integration** - a numerical method commonly used in real-time applications and molecular dynamics.

Unlike typical velocity-based position calculation, Verlet integration computes the **next position** from the **current position** and **previous position**.

Derivation Starting from Newton's equation of motion:

$$m \frac{d^2x(t)}{dt^2} = F$$

Using Taylor expansions:

$$x(t + \Delta t) = x(t) + \Delta t \frac{dx(t)}{dt} + \frac{1}{2} \Delta t^2 \frac{d^2 x(t)}{dt^2} + \dots$$

$$x(t - \Delta t) = x(t) - \Delta t \frac{dx(t)}{dt} + \frac{1}{2} \Delta t^2 \frac{d^2 x(t)}{dt^2} - \dots$$

Adding these equations and solving for the next position:

$$x(t + \Delta t) = 2x(t) - x(t - \Delta t) + \frac{\Delta t^2}{m} F(t)$$

Velocity is approximated as:

$$v(t) = \frac{x(t) - x(t - \Delta t)}{\Delta t}$$

This velocity approximation isn't highly accurate but suffices for spring damping calculations.

Sphere Collision

Collision handling has two phases: **detection** and **response**.

Detection

$$\|x(t + \Delta t) - c\| - r < 0$$

Where c is the sphere center and r is the radius.

Response If collision is detected, move the mass point to the sphere surface:

$$d = \frac{x(t + \Delta t) - c}{\|x(t + \Delta t) - c\|}$$

$$x'(t + \Delta t) = c + dr$$

The direction d approximates the surface normal at the collision point.

Implementation

Architecture Overview

GPUClothSimulation.cs

- └─ Creates/manages RenderTextures (position, previous position, normal)
- └─ Calls Kernels.compute kernels
- └─ Handles simulation loop

Kernels.compute

- └─ CSInit - Initialize buffers
- └─ CSSimulation - Main simulation step

GPUClothRenderer.cs

└─ Generates mesh, applies shader

ClothSurface.shader

└─ Deforms mesh using simulation data

GPUClothSimulation.cs

```
public class GPUClothSimulation : MonoBehaviour
{
    [Header("Simulation Parameters")]
    public float TimeStep = 0.01f;
    [Range(1, 16)]
    public int VerletIterationNum = 4;
    public Vector2Int ClothResolution = new Vector2Int(128, 128);
    public float RestLength = 0.02f;
    public float Stiffness = 10000.0f;
    public float Damp = 0.996f;
    public float Mass = 1.0f;
    public Vector3 Gravity = new Vector3(0.0f, -9.81f, 0.0f);

    [Header("References")]
    public Transform CollisionSphereTransform;
    public ComputeShader KernelCS;

    // Simulation buffers
    private RenderTexture[] _posBuff;           // Position (ping-pong)
    private RenderTexture[] _posPrevBuff;       // Previous position (ping-pong)
    private RenderTexture _normBuff;           // Normals

    const int numThreadsXY = 32;

    void Start()
    {
        var format = RenderTextureFormat.ARGBFloat; // 32-bit per channel
        var filter = FilterMode.Point; // No interpolation

        CreateRenderTexture(ref _posBuff, w, h, format, filter);
        CreateRenderTexture(ref _posPrevBuff, w, h, format, filter);
        CreateRenderTexture(ref _normBuff, w, h, format, filter);

        ResetBuffer();
        IsInit = true;
    }

    void Simulation()
    {
        float timestep = TimeStep / VerletIterationNum;

        // Set parameters
        cs.SetVector("_Gravity", Gravity);
        cs.SetFloat("_Stiffness", Stiffness);
        cs.SetFloat("_Damp", Damp);
    }
}
```

```

cs.SetFloat("_InverseMass", 1.0f / Mass);
cs.SetFloat("_TimeStep", timestep);

// Collision sphere parameters
if (CollisionSphereTransform != null)
{
    Vector3 pos = CollisionSphereTransform.position;
    float rad = CollisionSphereTransform.localScale.x * 0.5f + 0.01f;
    cs.SetBool("_EnableCollideSphere", true);
    cs.SetFloats("_CollideSphereParams",
        new float[] { pos.x, pos.y, pos.z, rad });
}

// Multiple iterations for stability
for (var i = 0; i < VerletIterationNum; i++)
{
    cs.SetTexture(kernelId, "_PositionBufferR0", _posBuff[0]);
    cs.SetTexture(kernelId, "_PositionPrevBufferR0", _posPrevBuff[0]);
    cs.SetTexture(kernelId, "_PositionBufferRW", _posBuff[1]);
    cs.SetTexture(kernelId, "_PositionPrevBufferRW", _posPrevBuff[1]);
    cs.SetTexture(kernelId, "_NormalBufferRW", _normBuff);

    cs.Dispatch(kernelId, groupThreadsX, groupThreadsY, 1);

    // Ping-pong buffer swap
    SwapBuffer(ref _posBuff[0], ref _posBuff[1]);
    SwapBuffer(ref _posPrevBuff[0], ref _posPrevBuff[1]);
}
}
}

```

Kernels.compute

```

#pragma kernel CSInit
#pragma kernel CSSimulation

#define NUM_THREADS_XY 32

// Buffers
Texture2D<float4> _PositionPrevBufferR0;
Texture2D<float4> _PositionBufferR0;
RWTexture2D<float4> _PositionPrevBufferRW;
RWTexture2D<float4> _PositionBufferRW;
RWTexture2D<float4> _NormalBufferRW;

// Parameters
int2 _ClothResolution;
float2 _TotalClothLength;
float _RestLength;
float3 _Gravity;
float _Stiffness;
float _Damp;
float _InverseMass;
float _TimeStep;

```

```

bool _EnableCollideSphere;
float4 _CollideSphereParams; // xyz = position, w = radius

// Neighbor offsets (12 connections)
static const int2 m_Directions[12] =
{
    int2(-1, -1), int2( 0, -1), int2( 1, -1), int2( 1,  0),
    int2( 1,  1), int2( 0,  1), int2(-1,  1), int2(-1,  0),
    int2(-2, -2), int2( 2, -2), int2( 2,  2), int2(-2,  2)
};

// Initialization kernel
[numthreads(NUM_THREADS_XY, NUM_THREADS_XY, 1)]
void CSInit(uint3 DTid : SV_DispatchThreadID)
{
    uint2 idx = DTid.xy;

    // Position: grid layout on X-Y plane
    float3 pos = float3(idx.x * _RestLength, idx.y * _RestLength, 0);
    pos.xy -= _TotalClothLength.xy * 0.5;

    // Normal: facing -Z
    float3 nrm = float3(0, 0, -1);

    _PositionPrevBufferRW[idx] = float4(pos, 1.0);
    _PositionBufferRW[idx] = float4(pos, 1.0);
    _NormalBufferRW[idx] = float4(nrm, 1.0);
}

// Simulation kernel
[numthreads(NUM_THREADS_XY, NUM_THREADS_XY, 1)]
void CSSimulation(uint2 DTid : SV_DispatchThreadID)
{
    int2 idx = (int2)DTid.xy;
    int2 res = _ClothResolution.xy;

    // Read current and previous positions
    float3 pos = _PositionBufferR0[idx].xyz;
    float3 posPrev = _PositionPrevBufferR0[idx].xyz;

    // Calculate velocity from position difference
    float3 vel = (pos - posPrev) / _TimeStep;

    float3 normal = (float3)0;
    float3 lastDiff = (float3)0;
    float iters = 0.0;

    // Initialize force with gravity
    float3 force = _Gravity.xyz;
    float invMass = _InverseMass;

    // Fix top edge (no simulation)
    if (idx.y == _ClothResolution.y - 1)
        return;

```

```

// Process all 12 neighbors
[unroll]
for (int k = 0; k < 12; k++)
{
    int2 neighCoord = m_Directions[k];

    // Skip if neighbor is outside bounds
    if (((idx.x + neighCoord.x) < 0) ||
        ((idx.x + neighCoord.x) > (res.x - 1)))
        continue;
    if (((idx.y + neighCoord.y) < 0) ||
        ((idx.y + neighCoord.y) > (res.y - 1)))
        continue;

    int2 idxNeigh = idx + neighCoord;
    float3 posNeigh = _PositionBufferR0[idxNeigh].xyz;
    float3 posDiff = posNeigh - pos;

    // Normal calculation using cross products
    float3 currDiff = normalize(posDiff);
    if ((iters > 0.0) && (k < 8))
    {
        float a = dot(currDiff, lastDiff);
        if (a > 0.0) {
            normal += cross(lastDiff, currDiff);
        }
    }
    lastDiff = currDiff;

    // Spring force calculation
    float restLength = length(neighCoord * _RestLength);
    force += (currDiff * (length(posDiff) - restLength)) * _Stiffness
            - vel * _Damp;

    if (k < 8) iters += 1.0;
}

// Finalize normal
normal = normalize(normal / -(iters - 1.0));

// Calculate acceleration (F = ma, a = F/m)
float3 acc = force * invMass;

// Verlet integration
float3 tmp = pos;
pos = pos * 2.0 - posPrev + acc * (_TimeStep * _TimeStep);
posPrev = tmp;

// Sphere collision
if (_EnableCollideSphere)
{
    float3 center = _CollideSphereParams.xyz;
    float radius = _CollideSphereParams.w;

```



```

        if (length(pos - center) < radius)
        {
            float3 collDir = normalize(pos - center);
            pos = center + collDir * radius;
        }

// Write results
_PositionBufferRW[idx] = float4(pos, 1.0);
_PositionPrevBufferRW[idx] = float4(posPrev, 1.0);
_NormalBufferRW[idx] = float4(normal, 1.0);
}

```

Parameters Guide

Parameter	Description	Effect
TimeStep	Simulation time per Update	Larger = faster motion, but may become unstable
VerletIterationNum	Kernel calls per frame	Larger = more stable, but higher compute cost
ClothResolution	Grid size (particles)	Larger = more detail, but may destabilize. Use multiples of 32.
RestLength	Spring natural length	Cloth size = Resolution x RestLength
Stiffness	Spring hardness	Larger = less stretching, but may destabilize
Damp	Velocity damping	Larger = less oscillation, slower movement
Mass	Particle mass	Larger = heavier, more dramatic movement
Gravity	Gravitational force	Combined with spring force for acceleration

Tips

- Press **R** key to reset simulation
 - **EnableDebugOnGUI** shows position/normal textures on screen
 - If simulation explodes, reset and adjust parameters
-

Ping-Pong Buffering

The simulation uses **ping-pong buffering** - maintaining two sets of buffers: 1. Read from buffer A, write to buffer B 2. Swap buffers 3. Read from buffer B (now containing new data), write to buffer A 4. Repeat

This pattern is essential for GPU computation where reading and writing to the same buffer simultaneously causes race conditions.

Key Takeaways

1. **Mass-Spring Systems** model elastic bodies using connected mass points
2. **Verlet Integration** calculates position from current and previous positions (no velocity storage)
3. **12-neighbor connectivity** provides structure (horizontal/vertical) and shear (diagonal) springs
4. **Ping-pong buffering** enables safe GPU read/write operations
5. **Multiple iterations per frame** improve stability with smaller effective timesteps
6. **Simple sphere collision** projects penetrating points to sphere surface

Future Directions

This chapter covers basic cloth simulation. Advanced topics include: - Complex geometry collision (not just spheres) - Cloth self-collision - Friction simulation - Fiber structure modeling - Large timestep stability improvements

References

- Marco Fratarcangeli, “Game Engine Gems 2, GPGPU Cloth simulation using GLSL, OpenCL, and CUDA”
- Wikipedia - Verlet integration
- Makoto Fujisawa, “Fundamentals of Physical Simulation for CG”
- Koichi Sakai, “Physical Simulation with WebGL”
- Koichi Sakai, “Introduction to Dynamics Animation with OpenGL”

Chapter 4: StarGlow

Author: Xjine **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** StarGlow

Introduction

When strong light reflects, it often produces rays extending outward. This effect goes by many names - Light Leak, Light Streak, or StarGlow. This chapter demonstrates how to create this effect as a post-process. We'll call it **StarGlow**.

This post-effect was presented by Masaki Kawase at GDC 2003 and remains an elegant technique for achieving dramatic lighting effects.

Overview

The StarGlow effect works in four main steps:

1. **Extract brightness** - Create a texture containing only bright pixels
 2. **Directional blur** - Apply streaking blur in specific directions
 3. **Composite streaks** - Combine multiple blur directions
 4. **Final composite** - Blend result with original image
-

STEP 1: Generate Brightness Image

First, detect and extract only the bright parts of the image. This is similar to what's done for general glow/bloom effects.

C# Setup

```
// Create reduced-resolution RenderTexture for performance
RenderTexture brightnessTex = RenderTexture.GetTemporary(
    source.width / this.divide,
    source.height / this.divide,
    source.depth,
    source.format
);

// Pass parameters to shader
// x = threshold, y = intensity, z = attenuation
material.SetVector(this.idParameter,
    new Vector3(threshold, intensity, attenuation));

Graphics.Blit(source, brightnessTex, material, 1); // Pass 1
```

Shader (Pass 1)

```
#define BRIGHTNESS_THRESHOLD _Parameter.x
#define INTENSITY            _Parameter.y
#define ATTENUATION          _Parameter.z

fixed4 frag(v2f_img input) : SV_Target
{
    float4 color = tex2D(_MainTex, input.uv);
    return max(color - BRIGHTNESS_THRESHOLD, 0) * INTENSITY;
}
```

How It Works

- **BRIGHTNESS_THRESHOLD** - Cutoff value; pixels below this become black
- **INTENSITY** - Multiplier for surviving brightness values
- **ATTENUATION** - Used later for blur decay

Brighter pixels produce larger values. Higher thresholds reduce the number of visible bright spots. Higher intensity makes the glow stronger.

Critical Optimization

The brightness texture is created at reduced resolution. Post-effects scale with pixel count - higher resolution means more fragment shader calls. Since glow involves iterative processing, this cost multiplies. Reducing resolution significantly improves performance with minimal visual impact.

STEP 2: Apply Directional Blur

Apply a blur that stretches in a specific direction, creating the characteristic star rays rather than the omnidirectional blur of typical glow.

C# Setup

```
Vector2 offset = new Vector2(-1, -1); // Blur direction
// In practice: Quaternion.AngleAxis(angle, Vector3.forward) * Vector2.down

material.SetVector(this.idOffset, offset);
material.SetInt(this.idIteration, 1);

Graphics.Blit(brightnessTex, blurredTex1, material, 2); // Pass 2

// Repeat for additional stretching
for (int i = 2; i <= this.iteration; i++)
{
    material.SetInt(this.idIteration, i);
    Graphics.Blit(blurredTex1, blurredTex2, material, 2);

    // Swap buffers
    RenderTexture temp = blurredTex1;
    blurredTex1 = blurredTex2;
    blurredTex2 = temp;
}
```

Shader (Pass 2)

```
int    _Iteration;
float2  _Offset;

struct v2f_starglow
{
    float4 pos      : SV_POSITION;
    float2 uv       : TEXCOORD0;
    half   power    : TEXCOORD1;
    half2  offset   : TEXCOORD2;
};

v2f_starglow vert(appdata_img v)
{
    v2f_starglow o;
    o.pos = UnityObjectToClipPos(v.vertex);
    o.uv = v.texcoord;

    // Power increases exponentially with iteration
    o.power = pow(4, _Iteration - 1);

    // Offset scales with pixel size and power
    o.offset = _MainTex_TexelSize.xy * _Offset * o.power;

    return o;
}

float4 frag(v2f_starglow input) : SV_Target
{
    half4 color = half4(0, 0, 0, 0);
    half2 uv = input.uv;
```

```

// Sample 4 pixels along the offset direction
for (int j = 0; j < 4; j++)
{
    color += saturate(tex2D(_MainTex, uv)
        * pow(ATTENUATION, input.power * j));
    uv += input.offset;
}

return color;
}

```

Understanding the Blur

Vertex Shader: - $\text{power} = 4^{(\text{iteration}-1)}$ - Controls how far to sample - $\text{offset} = \text{pixel size} \times \text{direction} \times \text{power}$ - The step between samples

Calculating these in the vertex shader avoids redundant computation in the fragment shader.

Fragment Shader: - Samples 4 pixels along the offset direction - Each sample is attenuated: sample 0 gets ATTENUATION^0 , sample 1 gets ATTENUATION^1 , etc. - Results are accumulated

Example with ATTENUATION = 0.7: - Sample 0: $x \cdot 0.7^0 = x \cdot 1.0$ - Sample 1: $x \cdot 0.7^1 = x \cdot 0.7$ - Sample 2: $x \cdot 0.7^2 = x \cdot 0.49$ - Sample 3: $x \cdot 0.7^3 = x \cdot 0.34$

This creates a gradient that fades along the streak direction.

Iterative Stretching

Each iteration: 1. Takes the already-blurred image as input 2. Increases power, sampling further away 3. Produces progressively longer streaks

The 4-sample count per pass is from Kawase's original presentation. Typically 2-3 total iterations produce good results while remaining performant.

STEP 2.5: Composite Multiple Directions

For a proper star effect, we need streaks in multiple directions. We repeat the blur process with different offset directions and composite the results.

C# Implementation

```

for (int x = 1; x <= this.numOfStreak; x++)
{
    // Calculate offset direction for this streak
    Vector2 offset = Quaternion.AngleAxis(
        angle * x + this.angleOfStreak,
        Vector3.forward
    ) * Vector2.down;
    offset = offset.normalized;

    // Apply iterative blur for this direction
    // ...

    // Composite to accumulated texture

```

```

    Graphics.Blit(blurredTex1, compositeTex, material, 3); // Pass 3
}

```

Shader (Pass 3) - Soft Blending

Blend OneMinusDstColor One

```

fixed4 frag(v2f_img input) : SV_Target
{
    return tex2D(_MainTex, input.uv);
}

```

The blending mode OneMinusDstColor One creates a soft, screen-like blend. This can be customized based on desired artistic effect.

STEP 3: Final Composite

Blend the accumulated streak texture with the original image.

C# Setup

```

// Enable selected composite mode
material.EnableKeyword(StarGlow.CompositeTypes[this.compositeType]);
material.SetColor(this.idCompositeColor, this.color);
material.SetTexture(this.idCompositeTex, compositeTex);

```

```

Graphics.Blit(source, destination, material, 4); // Pass 4

```

Shader (Pass 4) - Multiple Composite Modes

```

#pragma multi_compile _COMPOSITE_TYPE_ADDITIVE _COMPOSITE_TYPE_SCREEN ...

```

```

fixed4 frag(v2f_img input) : SV_Target
{
    float4 mainColor = tex2D(_MainTex, input.uv);
    float4 compositeColor = tex2D(_CompositeTex, input.uv);

    // Optional color tinting
    #if defined(_COMPOSITE_TYPE_COLORED_ADDITIVE) \
        || defined(_COMPOSITE_TYPE_COLORED_SCREEN)
        compositeColor.rgb =
            (compositeColor.r + compositeColor.g + compositeColor.b)
            * 0.3333 * _CompositeColor;
    #endif

    // Screen blend
    #if defined(_COMPOSITE_TYPE_SCREEN) \
        || defined(_COMPOSITE_TYPE_COLORED_SCREEN)
        return saturate(mainColor + compositeColor
            - saturate(mainColor * compositeColor));
    #endif

    // Additive blend
    #elif defined(_COMPOSITE_TYPE_ADDITIVE) \

```

```

    || defined(_COMPOSITE_TYPE_COLORED_ADDITIVE)
    return saturate(mainColor + compositeColor);

    // Debug: show only streaks
    #else
    return compositeColor;
    #endif
}

```

This provides flexibility for: - **Additive** - Simple brightness addition - **Screen** - Softer, more natural blend - **Colored variants** - Apply tint color to streaks

STEP 4: Resource Cleanup

Always release temporary resources:

```

material.DisableKeyword(StarGlow.CompositeTypes[this.compositeType]);

RenderTexture.ReleaseTemporary(brightnessTex);
RenderTexture.ReleaseTemporary(blurredTex1);
RenderTexture.ReleaseTemporary(blurredTex2);
RenderTexture.ReleaseTemporary(compositeTex);

```

Parameters Summary

Parameter	Description	Tips
Threshold	Brightness cutoff	Higher = fewer glowing areas
Intensity	Brightness multiplier	Higher = stronger glow
Attenuation	Streak fade rate	Lower = faster fade, shorter streaks
Divide	Resolution divisor	Higher = better performance, softer result
Iteration	Blur passes per direction	2-3 typical; more = longer streaks
NumOfStreak	Number of ray directions	4 = classic cross, 6 = star of David
AngleOfStreak	Rotation offset	Adjust starting angle
Color	Tint color	Warm colors for sun, cool for artificial light

Performance Considerations

This effect is relatively expensive because: 1. Multiple shader passes per frame 2. Multiple streak directions, each with iterations 3. Fragment-heavy operations

Optimization strategies: - Reduce resolution (divide parameter) - Limit iteration count - Reduce number of streaks - Consider temporal spreading (alternate directions across frames)

Variations and Extensions

Beyond the basic Kawase implementation:

- **Non-uniform streaks** - Vary parameters per iteration for organic, asymmetric rays
- **Animated parameters** - Use noise to vary parameters over time for shimmer
- **Threshold variation** - Different thresholds can create multiple glow layers
- **Colored streaks** - Different colors per direction for chromatic effects

For physically accurate light rays, you'd need approaches beyond post-processing (volumetric rendering, light shafts), but StarGlow provides an efficient, visually appealing approximation that works well for stylized rendering.

Key Takeaways

1. **Brightness extraction** isolates pixels above threshold for processing
2. **Resolution reduction** is critical for performance in iterative effects
3. **Directional blur** with attenuation creates the streak effect
4. **Exponential sampling** ($\text{pow}(4, \text{iteration})$) efficiently extends streak length
5. **Multiple directions** combined create the star pattern
6. **Flexible compositing** allows artistic control over final appearance

References

- Frame Buffer Postprocessing Effects in DOUBLE-S.T.E.A.L (Wreckless) - Masaki Kawase, GDC 2003
– http://www.daionet.gr.jp/~masa/archives/GDC2003_DSTEAL.ppt

Chapter 5: Triangulation by Ear Clipping

Author: Kaiware007 **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** TriangulationByEarClipping

Introduction

This chapter explains **Ear Clipping** - a method for triangulating polygons. We cover not only simple polygon triangulation but also polygons with holes and hierarchically nested polygons.

Sample Instructions

Run the DrawTest scene: - **Left click** - Place vertices to create a polygon (avoid self-intersecting lines) - **Right click** - Triangulate and generate mesh - Create polygons inside existing meshes to create holes

Simple Polygon Triangulation

A **simple polygon** is a closed polygon whose edges do not self-intersect.

Key property: Any simple polygon with n vertices can be triangulated into exactly $n-2$ triangles.

The Ear Clipping Algorithm

Among various triangulation methods, Ear Clipping is notable for its simplicity. It's based on the **Two Ears Theorem**:

Any simple polygon with 4 or more vertices has at least two "ears."

What is an "Ear"?

An **ear** is a triangle where: - Two edges are edges of the polygon - The third edge lies entirely inside the polygon

Algorithm Concept

Ear Clipping works by: 1. Finding ears in the polygon 2. Removing (clipping) them one at a time 3. Repeating until only one triangle remains

Note: While simple to implement, Ear Clipping is slower than other algorithms and may not be suitable for performance-critical applications.

Triangulation Process

Ear Detection Criteria

A vertex v_i is an ear if:

1. **Convex vertex** - The interior angle at v_i (formed by v_{i-1} , v_i , v_{i+1}) is less than 180 degrees
2. **No contained vertices** - The triangle formed by v_{i-1} , v_i , v_{i+1} contains no other polygon vertices

Step-by-Step Process

Initialization: 1. Check all vertices against ear criteria 2. Add qualifying vertices to the ear list

Main Loop: 1. Take the first vertex from the ear list 2. Create a triangle with its neighboring vertices 3. Remove the vertex from the polygon 4. Re-check the two neighboring vertices (they may become ears or stop being ears) 5. Repeat until only 3 vertices remain (final triangle)

Worked Example

Consider a 7-vertex polygon (vertices 0-6):

Initial state: - Ear list: [0, 1, 4, 6] - Vertex 2, 5: Not convex (angle ≥ 180) - Vertex 3: Triangle (2,3,4) contains vertex 5

Iteration 1: Remove vertex 0 - Create triangle (6, 0, 1) - Vertices 1, 6 remain ears - Ear list: [1, 4, 6]

Iteration 2: Remove vertex 1 - Create triangle (6, 1, 2) - Vertex 2 becomes convex and is now an ear - Ear list: [4, 6, 2]

Iteration 3: Remove vertex 4 - Create triangle (3, 4, 5) - Vertices 3, 5 become ears - Ear list: [6, 2, 3, 5]

Continue until complete...

Polygons with Holes

Ear Clipping doesn't directly handle holes, but we can work around this by **connecting** the outer polygon to inner polygons, creating a single simple polygon.

Connection Process

Prerequisites: - Outer polygon vertices are ordered clockwise - Inner polygon vertices are ordered counter-clockwise

Steps:

1. **Find rightmost inner vertex** - Among all hole polygons, find the one with the maximum X-coordinate vertex. Call this vertex M.
2. **Cast ray right** - From M, cast a ray in the +X direction.
3. **Find intersection** - Find where this ray intersects the outer polygon. Call the intersection point I and the edge endpoints.
4. **Select connection point P** - Choose the edge endpoint with the larger X-coordinate.
5. **Check for obstructions** - Form triangle M-I-P. If no other vertices are inside this triangle, P is the connection point. If vertices exist inside, choose the one with the smallest angle to line M-I.
6. **Connect polygons** - Insert the inner polygon's vertices into the outer polygon at point P, duplicating M and P to create the "bridge."
7. **Repeat** - Process remaining holes from right to left.

Why Right to Left?

Processing holes from rightmost to leftmost ensures that previously created bridges don't interfere with subsequent connections.

Nested Polygon Hierarchies

For polygons containing polygons (like islands within lakes within land), we need to establish parent-child relationships.

Building the Hierarchy

1. **Sort by bounding box area** - Larger polygons are potential parents
2. **Build tree recursively** - For each polygon, check if it's contained within another
3. **Alternate winding** - Odd depth = counter-clockwise (outer), even depth = clockwise (inner)
4. **Create dummy root** - Handle multiple independent polygons

Processing the Hierarchy

1. Take a polygon from the tree (this is the outer polygon)
 2. Get all immediate children (these are holes)
 3. Connect and triangulate as a polygon-with-holes
 4. Repeat for remaining polygons in the tree
-

Implementation

Data Structures

```
// Vertex coordinates
List<Vector3> vertices = new List<Vector3>();

// Vertex indices (circular linked list behavior)
LinkedList<int> indices = new LinkedList<int>();

// Current ear vertices
List<int> earTipList = new List<int>();
```

Polygon Class

```
public class Polygon
{
    public enum LoopType
    {
        CW,    // Clockwise
        CCW,   // Counter-clockwise
        ERR,   // Undefined
    }

    public Vector3[] vertices;
    public LoopType loopType;
    // Methods for containment testing, etc.
}
```

Tree Structure

```
public class TreeNode<T>
{
    public TreeNode<T> parent = null;
    public List<TreeNode<T>> children = new List<TreeNode<T>>();
    public T Value;
    public bool isValue = false; // False for dummy root

    public void AddChild(T val) { ... }
    public void RemoveChild(TreeNode<T> tree) { ... }
}
```

Ear Checking

```
void CheckVertex(LinkedListNode<int> node)
{

```

```

int prevIndex = (node.Previous == null) ?
    indices.Last.Value : node.Previous.Value;
int nextIndex = (node.Next == null) ?
    indices.First.Value : node.Next.Value;
int nowIndex = node.Value;

Vector3 prevVertex = vertices[prevIndex];
Vector3 nextVertex = vertices[nextIndex];
Vector3 nowVertex = vertices[nowIndex];

bool isEar = false;

// Check if interior angle < 180 degrees
if (GeomUtil.IsAngleLessPI(nowVertex, prevVertex, nextVertex))
{
    isEar = true;

    // Check no other vertices inside triangle
    foreach (int i in indices)
    {
        if (i == prevIndex || i == nowIndex || i == nextIndex)
            continue;

        Vector3 p = vertices[i];

        // Skip duplicated vertices (from hole connections)
        if (Vector3.Distance(p, prevVertex) <= float.Epsilon) continue;
        if (Vector3.Distance(p, nowVertex) <= float.Epsilon) continue;
        if (Vector3.Distance(p, nextVertex) <= float.Epsilon) continue;

        if (GeomUtil.IsInTriangle(p, prevVertex, nowVertex, nextVertex) <= 0)
        {
            isEar = false;
            break;
        }
    }

    // Update ear list
    if (isEar && !earTipList.Contains(nowIndex))
        earTipList.Add(nowIndex);
    else if (!isEar && earTipList.Contains(nowIndex))
        earTipList.Remove(nowIndex);
}
}

```

Point-in-Triangle Test

```

// Check which side of a line a point is on
public static int CheckLine(Vector3 o, Vector3 p1, Vector3 p2)
{
    double x0 = o.x - p1.x;
    double y0 = o.y - p1.y;
    double x1 = p2.x - p1.x;
    double y1 = p2.y - p1.y;

```

```

    double det = x0 * y1 - x1 * y0; // 2D cross product

    return (det > 0 ? +1 : (det < 0 ? -1 : 0));
    // +1 = right, -1 = left, 0 = on line
}

// Triangle containment (clockwise winding)
public static int IsInTriangle(Vector3 o, Vector3 p1, Vector3 p2, Vector3 p3)
{
    int sign1 = CheckLine(o, p2, p3);
    if (sign1 < 0) return +1; // Outside

    int sign2 = CheckLine(o, p3, p1);
    if (sign2 < 0) return +1;

    int sign3 = CheckLine(o, p1, p2);
    if (sign3 < 0) return +1;

    // All same sign = inside, any zero = on edge
    return ((sign1 != 0 && sign2 != 0 && sign3 != 0) ? -1 : 0);
}

```

Main Ear Clipping Loop

```

void EarClipping()
{
    while (earTipList.Count > 0)
    {
        int nowIndex = earTipList[0]; // Take first ear

        LinkedListNode<int> indexNode = indices.Find(nowIndex);
        if (indexNode != null)
        {
            int prevIndex = (indexNode.Previous == null) ?
                indices.Last.Value : indexNode.Previous.Value;
            int nextIndex = (indexNode.Next == null) ?
                indices.First.Value : indexNode.Next.Value;

            // Create triangle
            resultTriangulation.Add(prevIndex);
            resultTriangulation.Add(nowIndex);
            resultTriangulation.Add(nextIndex);

            if (indices.Count == 3)
                break; // Final triangle

            // Remove ear vertex
            earTipList.RemoveAt(0);
            indices.Remove(nowIndex);

            // Re-check neighbors
            CheckVertex(indices.Find(prevIndex));
            CheckVertex(indices.Find(nextIndex));
        }
    }
}

```

```

    }
}
}

```

Mesh Generation

```

void MakeMesh()
{
    mesh = new Mesh();
    mesh.vertices = resultVertices.ToArray();
    mesh.SetIndices(resultTriangulation.ToArray(),
                    MeshTopology.Triangles, 0);
    mesh.RecalculateNormals();
    mesh.SetUVs(0, resultUVs);
    mesh.RecalculateBounds();

    GetComponent<MeshFilter>().mesh = mesh;
}

```

Geometry Utilities Summary

Function	Purpose
IsAngleLessPI	Check if interior angle < 180 degrees
CheckLine	Determine which side of a line a point is on
IsInTriangle	Point-in-triangle test
GetIntersectionPoint	Find line-polygon intersection
IsIntersectLine	Check if two line segments intersect

Key Takeaways

1. **Ear Clipping** is conceptually simple but not the fastest triangulation algorithm
2. **Two Ears Theorem** guarantees at least 2 ears exist in any simple polygon
3. **Ear criteria**: Convex vertex + empty triangle interior
4. **Holes** are handled by connecting inner polygons to outer via bridges
5. **Nested polygons** require hierarchy construction with alternating winding orders
6. **LinkedList** provides efficient vertex removal during the algorithm
7. **2D cross product** is key for point-line orientation tests

Applications

- Real-time mesh generation from user-drawn shapes
- Font outline to mesh conversion
- Procedural level geometry
- Any polygon-to-triangles conversion need

Performance Note

Ear Clipping has $O(n^2)$ time complexity in the worst case. For performance-critical applications with many vertices, consider faster algorithms like: - Monotone polygon decomposition + triangulation - Delaunay triangulation - Constrained Delaunay triangulation

References

- Triangulation by Ear Clipping - <https://www.geometrictools.com/Documentation/TriangulationByEarClipping.pdf>
- Wikipedia: Polygon triangulation

Chapter 6: Tessellation & Displacement

Author: Sakota **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** Tessellation

Introduction

This chapter explains **Tessellation** - the GPU feature for subdividing polygons - and how to use **Displacement Maps** to offset the resulting vertices.

Requirements

- ComputeShader support: Shader Model 5.0+
 - Tessellation Shader only: Shader Model 4.6+
 - Tested with Unity 2018.3.9
-

What is Tessellation?

Tessellation is a standard feature in modern rendering pipelines (DirectX, OpenGL, Metal) that subdivides polygons on the GPU.

Why Use Tessellation?

Traditionally, all vertex data is transferred from CPU to GPU. For high-polygon meshes, this CPU-GPU transfer becomes a bottleneck. Tessellation solves this by:

1. Processing a reduced-polygon mesh on the CPU
2. Subdividing it on the GPU
3. Using displacement maps to restore fine detail

This reduces bandwidth while maintaining visual quality.

Tessellation Pipeline Stages

Tessellation adds three stages to the rendering pipeline:

1. **Hull Shader** - Programmable: Defines subdivision method and level
2. **Tessellator** - Fixed-function: Performs the actual subdivision

3. Domain Shader - Programmable: Positions the new vertices

The pipeline flow becomes:

Vertex Shader -> Hull Shader -> Tessellator -> Domain Shader -> [Geometry Shader] -> Fragment Shader

Surface Shader Implementation

Unity provides convenient tessellation wrappers for Surface Shaders. This is the easiest way to get started.

Shader "Custom/TessellationDisplacement"

```
{
    Properties
    {
        _EdgeLength ("Edge length", Range(2,50)) = 15
        _MainTex ("Base (RGB)", 2D) = "white" {}
        _DispTex ("Disp Texture", 2D) = "black" {}
        _NormalMap ("Normalmap", 2D) = "bump" {}
        _Displacement ("Displacement", Range(0, 1.0)) = 0.3
        _Color ("Color", color) = (1,1,1,0)
        _SpecColor ("Spec color", color) = (0.5,0.5,0.5,0.5)
        _Specular ("Specular", Range(0, 1)) = 0
        _Gloss ("Gloss", Range(0, 1)) = 0
    }
    SubShader
    {
        Tags { "RenderType"="Opaque" }
        LOD 300

        CGPROGRAM
        // tessellate: specifies the tessellation function
        // vertex: specifies the displacement function (called in Domain Shader)
        #pragma surface surf BlinnPhong addshadow fullforwardshadows \
            vertex:disp tessellate:tessEdge nolightmap
        #pragma target 4.6
        #include "Tessellation.cginc"

        struct appdata
        {
            float4 vertex : POSITION;
            float4 tangent : TANGENT;
            float3 normal : NORMAL;
            float2 texcoord : TEXCOORD0;
        };

        sampler2D _DispTex;
        float _Displacement;
        float _EdgeLength;

        // Tessellation function - called per patch, not per vertex
        // Returns subdivision levels: xyz = edge factors, w = inside factor
        float4 tessEdge(appdata v0, appdata v1, appdata v2)
        {
```



```

// Tessellation.cginc provides helper functions:
// - UnityDistanceBasedTess: Based on camera distance
// - UnityEdgeLengthBasedTess: Based on edge length
// - UnityEdgeLengthBasedTessCull: Edge length + frustum culling

return UnityEdgeLengthBasedTessCull(
    v0.vertex, v1.vertex, v2.vertex,
    _EdgeLength, _Displacement * 1.5f
);
}

// Displacement function - called in Domain Shader after tessellation
void disp(inout appdata v)
{
    // Sample displacement map and offset along normal
    float d = tex2Dlod(
        _DispTex,
        float4(v.texcoord.xy, 0, 0)
    ).r * _Displacement;
    v.vertex.xyz += v.normal * d;
}

struct Input
{
    float2 uv_MainTex;
};

sampler2D _MainTex;
sampler2D _NormalMap;
fixed4 _Color;
float _Specular;
float _Gloss;

void surf(Input IN, inout SurfaceOutput o)
{
    half4 c = tex2D(_MainTex, IN.uv_MainTex) * _Color;
    o.Albedo = c.rgb;
    o.Specular = _Specular;
    o.Gloss = _Gloss;
    o.Normal = UnpackNormal(tex2D(_NormalMap, IN.uv_MainTex));
}
ENDCG
}
FallBack "Diffuse"
}

```

Built-in Tessellation Functions

Unity's Tessellation.cginc provides:

Function	Description
UnityDistanceBasedTess	More subdivision when closer to camera
UnityEdgeLengthBasedTess	Subdivision based on screen-space edge length

Function	Description
UnityEdgeLengthBasedTessCull	Edge-based + frustum culling optimization

Vertex/Fragment Shader Implementation

For more control, you can implement tessellation manually in vertex/fragment shaders.

Hull Shader

The Hull Shader runs after the Vertex Shader. It consists of two functions: - **Control Point Function** - Runs per control point (vertex) - **Patch Constant Function** - Runs per patch (primitive)

```
#pragma hull hull_shader

// Input structure for hull shader
struct InternalTessInterp_appdata
{
    float4 vertex : INTERNALTESSPOS;
    float4 tangent : TANGENT;
    float3 normal : NORMAL;
    float2 texcoord : TEXCOORD0;
};

// Tessellation factors returned by patch constant function
struct TessellationFactors
{
    float edge[3] : SV_TessFactor;      // Edge subdivision levels
    float inside : SV_InsideTessFactor; // Interior subdivision level
};

// Patch Constant Function - runs once per patch
TessellationFactors hull_const(InputPatch<InternalTessInterp_appdata, 3> v)
{
    TessellationFactors o;
    float4 tf;

    tf = UnityEdgeLengthBasedTessCull(
        v[0].vertex, v[1].vertex, v[2].vertex,
        _EdgeLength, _Displacement * 1.5f
    );

    o.edge[0] = tf.x;
    o.edge[1] = tf.y;
    o.edge[2] = tf.z;
    o.inside = tf.w;
    return o;
}

// Control Point Function - runs once per control point
[UNITY_domain("tri")] // Triangle primitives
[UNITY_partitioning("fractional_odd")] // Smooth subdivision
```

```

[UNITY_outputtopology("triangle_cw")] // Clockwise winding
[UNITY_patchconstantfunc("hull_const")] // Patch constant function name
[UNITY_outputcontrolpoints(3)] // 3 control points per patch
InternalTessInterp_appdata hull_shader(
    InputPatch<InternalTessInterp_appdata, 3> v,
    uint id : SV_OutputControlPointID
)
{
    return v[id];
}

```

Hull Shader Attributes

Attribute	Options	Description
UNITY_domain	"tri", "quad", "isoline"	Primitive type
UNITY_partitioning	"integer", "fractional_odd", "fractional_even"	Subdivision interpolation
UNITY_outputtopology	"triangle_cw", "triangle_ccw", "line"	Output winding/type
UNITY_patchconstantfunc	function name	Patch constant function
UNITY_outputcontrolpoints	number	Control points per patch

Tessellator Stage

This is the fixed-function stage. It reads the tessellation factors from the Hull Shader and subdivides the patch accordingly. No code is written for this stage.

Domain Shader

The Domain Shader positions vertices created by the Tessellator. It receives barycentric coordinates via SV_DomainLocation.

```
#pragma domain domain_shader
```

```

struct v2f
{
    UNITY_POSITION(pos);
    float2 uv_MainTex : TEXCOORD0;
    float4 tSpace0 : TEXCOORD1;
    float4 tSpace1 : TEXCOORD2;
    float4 tSpace2 : TEXCOORD3;
};

sampler2D _DispTex;
float _Displacement;

// Displacement function
void disp(inout appdata v)
{
    float d = tex2Dlod(_DispTex, float4(v.texcoord.xy, 0, 0)).r * _Displacement;
    v.vertex.xyz -= v.normal * d;
}

```

```

// Vertex processing (prepares data for fragment shader)
v2f vert_process(appdata v)
{
    v2f o;
    UNITY_INITIALIZE_OUTPUT(v2f, o);
    o.pos = UnityObjectToClipPos(v.vertex);
    o.uv_MainTex.xy = TRANSFORM_TEX(v.texcoord, _MainTex);

    float3 worldPos = mul(unity_ObjectToWorld, v.vertex).xyz;
    float3 worldNormal = UnityObjectToWorldNormal(v.normal);
    fixed3 worldTangent = UnityObjectToWorldDir(v.tangent.xyz);
    fixed tangentSign = v.tangent.w * unity_WorldTransformParams.w;
    fixed3 worldBinormal = cross(worldNormal, worldTangent) * tangentSign;

    o.tSpace0 = float4(worldTangent.x, worldBinormal.x, worldNormal.x, worldPos.x);
    o.tSpace1 = float4(worldTangent.y, worldBinormal.y, worldNormal.y, worldPos.y);
    o.tSpace2 = float4(worldTangent.z, worldBinormal.z, worldNormal.z, worldPos.z);

    return o;
}

// Domain Shader - positions tessellated vertices
[UNITY_domain("tri")]
v2f domain_shader(
    TessellationFactors tessFactors,
    const OutputPatch<InternalTessInterp_appdata, 3> vi,
    float3 bary : SV_DomainLocation
)
{
    appdata v;
    UNITY_INITIALIZE_OUTPUT(appdata, v);

    // Interpolate all attributes using barycentric coordinates
    v.vertex = vi[0].vertex * bary.x +
                vi[1].vertex * bary.y +
                vi[2].vertex * bary.z;

    v.tangent = vi[0].tangent * bary.x +
                vi[1].tangent * bary.y +
                vi[2].tangent * bary.z;

    v.normal = vi[0].normal * bary.x +
                vi[1].normal * bary.y +
                vi[2].normal * bary.z;

    v.texcoord = vi[0].texcoord * bary.x +
                 vi[1].texcoord * bary.y +
                 vi[2].texcoord * bary.z;

    // Apply displacement here
    disp(v);

    // Prepare for fragment shader
    v2f o = vert_process(v);

```

```

    return o;
}

```

Understanding SV_DomainLocation

The SV_DomainLocation semantic provides barycentric coordinates for triangle patches: - bary.x = weight for vertex 0 - bary.y = weight for vertex 1 - bary.z = weight for vertex 2 - Always: bary.x + bary.y + bary.z = 1.0

These coordinates are used to interpolate all vertex attributes (position, normal, UV, etc.) to the new tessellated vertex positions.

Application Example: Fluid Displacement

The chapter includes an example combining tessellation with fluid simulation from Volume 1:

1. GPU fluid simulation outputs a height map
2. Unity's built-in Plane mesh receives tessellation
3. Displacement shader samples height map and offsets vertices

The result: A simple plane mesh seamlessly follows complex fluid motion without mesh distortion, thanks to GPU-side subdivision.

Performance Considerations

Factor	Impact
Tessellation level	Higher = more triangles, more compute
Displacement texture sampling	Use tex2Dlod to avoid mipmap issues
Frustum culling	UnityEdgeLengthBasedTessCull skips off-screen patches
Edge-based subdivision	Prevents over-tessellation of small screen areas

Key Takeaways

1. **Tessellation** subdivides geometry on GPU, reducing CPU-GPU bandwidth
2. **Hull Shader** defines how and how much to subdivide (patch constants)
3. **Tessellator** is fixed-function - performs actual subdivision
4. **Domain Shader** positions new vertices using barycentric interpolation
5. **Surface Shader wrappers** provide easy integration for common cases
6. **Displacement** is applied in Domain Shader after tessellation
7. **Edge-length-based tessellation** adapts detail to screen-space size
8. **Frustum culling** optimizes by skipping invisible patches

Best Practices

- Use edge-length-based tessellation to maintain consistent screen-space detail
- Enable culling to skip off-screen geometry
- Balance tessellation level with performance requirements

- Combine with normal maps for detail that doesn't need geometric subdivision
- Use LOD systems for distance-based quality scaling

References

- Unity Manual: Surface Shader Tessellation
 - <https://docs.unity3d.com/Manual/SL-SurfaceShaderTessellation.html>
- Microsoft DirectX 11 Tessellation
 - <https://docs.microsoft.com/en-us/windows/desktop/direct3d11/direct3d-11-advanced-stages-tessellation>

Chapter 7: Poisson Disk Sampling

Author: Aoyama **Repository:** <https://github.com/IndieVisualLab/UnityGraphicsProgramming4> **Sample Project:** (Poisson Disk Sampling implementation)

Introduction

This chapter explains **Poisson Disk Sampling (PDS)** and its CPU implementation using the **Fast Poisson Disk Sampling in Arbitrary Dimensions (FPDS)** algorithm.

FPDS was proposed by Robert Bridson of the University of British Columbia in a SIGGRAPH 2007 paper. Its key innovation is dimension-independence - the same algorithm works in any number of dimensions.

What is Poisson Disk Sampling?

Consider plotting many points in a space with a minimum distance constraint **d** (where $d > 0$). A **Poisson-disk distribution** is a set of randomly positioned points where **every pair of points is separated by at least distance d**.

Key property: No matter which two points you select from the distribution, their distance will never be less than **d**.

PDS is the process of generating (sampling) such a distribution computationally.

Applications

PDS produces uniformly distributed random point sets, useful for: - **Anti-aliasing** - Sample positions for filtering - **Texture synthesis** - Selecting blend pixels - **Object placement** - Natural-looking random distributions - **Image effects** - Blur kernels, dithering - **Procedural generation** - Vegetation, crowd placement

Fast Poisson Disk Sampling Algorithm

Key Advantages of FPDS

- **Dimension-independent** - Same algorithm for 2D, 3D, or any dimension
- **O(N) time complexity** - Linear in the number of output points
- **Simple implementation** - Uses basic data structures

Previous PDS methods were typically limited to 2D. FPDS efficiently handles any dimension.

Parameters

Parameter	Symbol	Description
Sampling space	R^n	Bounding region for points
Minimum distance	r	Required separation between points
Sampling limit	k	Attempts per active point

Algorithm Steps

Step 1: Divide Space into Grid

To accelerate distance checks, divide the sampling space into a grid of cells.

Cell size: $r / \sqrt{n}$

Where n is the number of dimensions. For 3D: $r / \sqrt{3}$

Why this size? - $\sqrt{n}$ is the diagonal length of an n -dimensional unit hypercube - With this cell size, each cell can contain **at most one point** - Checking neighbors requires only looking at cells within $\pm n$ in each dimension

// Create 3D grid for sampling

```
Vector3?[,] GetGrid(Vector3 bottomLeftBack, Vector3 topRightForward,
    float min, int iteration)
{
    var dimension = (topRightForward - bottomLeftBack);
    var cell = min * InvertRootTwo; // r / sqrt(2) for 3D

    return new Vector3?[
        Mathf.CeilToInt(dimension.x / cell) + 1,
        Mathf.CeilToInt(dimension.y / cell) + 1,
        Mathf.CeilToInt(dimension.z / cell) + 1
    ];
}
```

// Get grid cell index for a point

```
Vector3Int GetGridIndex(Vector3 point, Settings set)
{
    return new Vector3Int(
        Mathf.FloorToInt((point.x - set.BottomLeftBack.x) / set.CellSize),
        Mathf.FloorToInt((point.y - set.BottomLeftBack.y) / set.CellSize),
        Mathf.FloorToInt((point.z - set.BottomLeftBack.z) / set.CellSize)
    );
}
```

Step 2: Create Initial Sample Point

Generate a completely random point within the sampling space. This is valid since no other points exist yet.

Add this point to: 1. Its corresponding grid cell 2. The **sample list** (final output) 3. The **active list** (points to expand from)

```

void GetFirstPoint(Settings set, Bags bags)
{
    var first = new Vector3(
        Random.Range(set.BottomLeftBack.x, set.TopRightForward.x),
        Random.Range(set.BottomLeftBack.y, set.TopRightForward.y),
        Random.Range(set.BottomLeftBack.z, set.TopRightForward.z)
    );
    var index = GetGridIndex(first, set);

    bags.Grid[index.x, index.y, index.z] = first;
    bags.SamplePoints.Add(first);
    bags.ActivePoints.Add(first);
}

```

Step 3: Select Base Point from Active List

Randomly select an index i from the active list. The point at this index, x_i , becomes the center for generating new samples.

```

var index = Random.Range(0, bags.ActivePoints.Count);
var point = bags.ActivePoints[index];

```

Step 4: Sample New Points

From x_i , generate a random point x'_i in the spherical shell between radius r and $2r$.

Why r to $2r$? - Less than r : Would violate minimum distance from x_i - Greater than $2r$: Leaves gaps that could contain valid points

Check if x'_i is valid: 1. Is it within the sampling bounds? 2. Is it at least distance r from all nearby points?

For the distance check, only examine cells within ± 3 of x'_i 's cell (covers all cells within distance r).

```

private static bool GetNextPoint(Vector3 point, Settings set, Bags bags)
{
    // Generate random point in spherical shell [r, 2r]
    var p = point + GetRandPosInSphere(set.MinimumDistance, 2f * set.MinimumDistance);

    // Check bounds
    if (!set.Dimension.Contains(p)) { return false; }

    var minimum = set.MinimumDistance * set.MinimumDistance;
    var index = GetGridIndex(p, set);
    var drop = false;

    // Calculate neighbor search range
    var around = 3;
    var fieldMin = new Vector3Int(
        Mathf.Max(0, index.x - around),
        Mathf.Max(0, index.y - around),
        Mathf.Max(0, index.z - around)
    );
    var fieldMax = new Vector3Int(
        Mathf.Min(set.GridWidth, index.x + around),
        Mathf.Min(set.GridHeight, index.y + around),
        Mathf.Min(set.GridDepth, index.z + around)
    );
}

```



```

);

// Check neighboring cells for conflicts
for (var i = fieldMin.x; i <= fieldMax.x && !drop; i++)
{
    for (var j = fieldMin.y; j <= fieldMax.y && !drop; j++)
    {
        for (var k = fieldMin.z; k <= fieldMax.z && !drop; k++)
        {
            var q = bags.Grid[i, j, k];
            if (q.HasValue && (q.Value - p).sqrMagnitude <= minimum)
            {
                drop = true; // Too close to existing point
            }
        }
    }
}

if (drop) { return false; }

// Point is valid - add to all data structures
bags.SamplePoints.Add(p);
bags.ActivePoints.Add(p);
bags.Grid[index.x, index.y, index.z] = p;
return true;
}

```

Step 5: Repeat Sampling

For each active point x_i , attempt k samples (Step 4).

If all k attempts fail (no valid points found), remove x_i from the active list - this area is fully sampled.

Return to Step 3 and select a new active point.

Continue until the active list is empty.

```

public static List<Vector3> Sampling(Vector3 bottomLeft, Vector3 topRight,
    float minimumDistance, int iterationPerPoint)
{
    var settings = GetSettings(
        bottomLeft, topRight, minimumDistance,
        iterationPerPoint <= 0 ? DefaultIterationPerPoint : iterationPerPoint
    );

    var bags = new Bags()
    {
        Grid = new Vector3?[
            settings.GridWidth + 1,
            settings.GridHeight + 1,
            settings.GridDepth + 1
        ],
        SamplePoints = new List<Vector3>(),
        ActivePoints = new List<Vector3>()
    };
}

```

```

    GetFirstPoint(settings, bags);

do
{
    var index = Random.Range(0, bags.ActivePoints.Count);
    var point = bags.ActivePoints[index];

    var found = false;
    for (var k = 0; k < settings.IterationPerPoint; k++)
    {
        found = found | GetNextPoint(point, settings, bags);
    }

    if (!found) { bags.ActivePoints.RemoveAt(index); }
}
while (bags.ActivePoints.Count > 0);

return bags.SamplePoints;
}

```

Algorithm Summary

1. **Divide space into grid** - Cell size $r/\sqrt{n}$ ensures one point per cell
 2. **Place initial random point** - Add to grid, sample list, and active list
 3. **Select active point** - Random selection from active list
 4. **Attempt k samples** - Generate points in $[r, 2r]$ shell, validate against neighbors
 5. **Remove exhausted points** - When k attempts yield no valid points
 6. **Repeat until done** - Active list empty = space fully sampled
-

Complexity Analysis

Operation	Complexity
Grid lookup	$O(1)$
Neighbor check	$O(1)$ - fixed number of cells
Per-point sampling	$O(k)$ - k attempts
Total	$O(N)$ - linear in output points

The grid structure is key - it transforms $O(N)$ distance checks into $O(1)$ neighbor lookups.

Visualization

The result can be visualized by placing circles/spheres at sampled positions: - No circles overlap (minimum distance maintained) - Space is filled as completely as possible - Distribution appears random but uniform

Parameter Guidelines

Parameter	Effect of Increasing
Space (R^n)	More points sampled (larger area)
Distance (r)	Fewer, more spread points
Attempts (k)	Better space filling, longer compute

Typical k values: 20-30 for good results without excessive computation.

Limitations

The basic FPDS algorithm: - **Not parallelized** - Sequential active list processing - **CPU-bound** - Large spaces or small r means significant computation time - **Memory** - Grid storage grows with space size / cell size

For real-time applications, consider: - Pre-computing samples - GPU-accelerated variants - Hierarchical approaches for large spaces

Key Takeaways

1. **Poisson Disk Sampling** produces uniformly distributed random points with minimum separation
2. **Grid acceleration** reduces distance checks from $O(N)$ to $O(1)$
3. **Cell size $r/\sqrt{n}$** guarantees at most one point per cell
4. **Spherical shell $[r, 2r]$** sampling ensures valid candidates
5. **Active list** tracks expansion frontier; removal when exhausted
6. **Dimension-independent** - Same algorithm for 2D, 3D, nD
7. **$O(N)$ complexity** makes it practical for moderate point counts
8. **Wide applications** - Anti-aliasing, object placement, procedural generation

Applications in Graphics

- **Sampling patterns** for ray tracing, ambient occlusion
- **Object scattering** for vegetation, debris, crowds
- **Stippling** and artistic rendering
- **Noise generation** with better spectral properties than white noise
- **Level-of-detail** point selection

References

- Fast Poisson Disk Sampling in Arbitrary Dimensions - Robert Bridson, SIGGRAPH 2007
 - <https://www.cct.lsu.edu/~fharhad/ganbatte/siggraph2007/CD2/content/sketches/0250.pdf>